🌐 Module 05: Web Application Penetration Testing

```
# Exercise 10: Pentesting SQL Injection Vulnerability to Extract Database Information
```

Scenario

In the previous lab exercise (**Exercise 05.9**), you performed a web application vulnerability assessment using OWASP ZAP, which identified a High-severity SQL Injection vulnerability. When penetration testers identify such vulnerabilities, they proceed to exploit them in order to demonstrate business impact and extract sensitive database contents.

As a Penetration Tester, you should have the expertise to pentest SQL injection vulnerabilities and extract sensitive records automatically using `sqlmap`.

This lab is a direct continuation of **Exercise 05.9**.

Objective

The objective of this lab is to help students learn how to:
- Utilize `sqlmap` with an extracted HTTP POST request payload
- Enumerate databases, tables, and column structures from a Microsoft SQL Server backend
- Dump and crack password hashes using automated dictionary attacks
- Validate compromised user credentials on the target web application

---

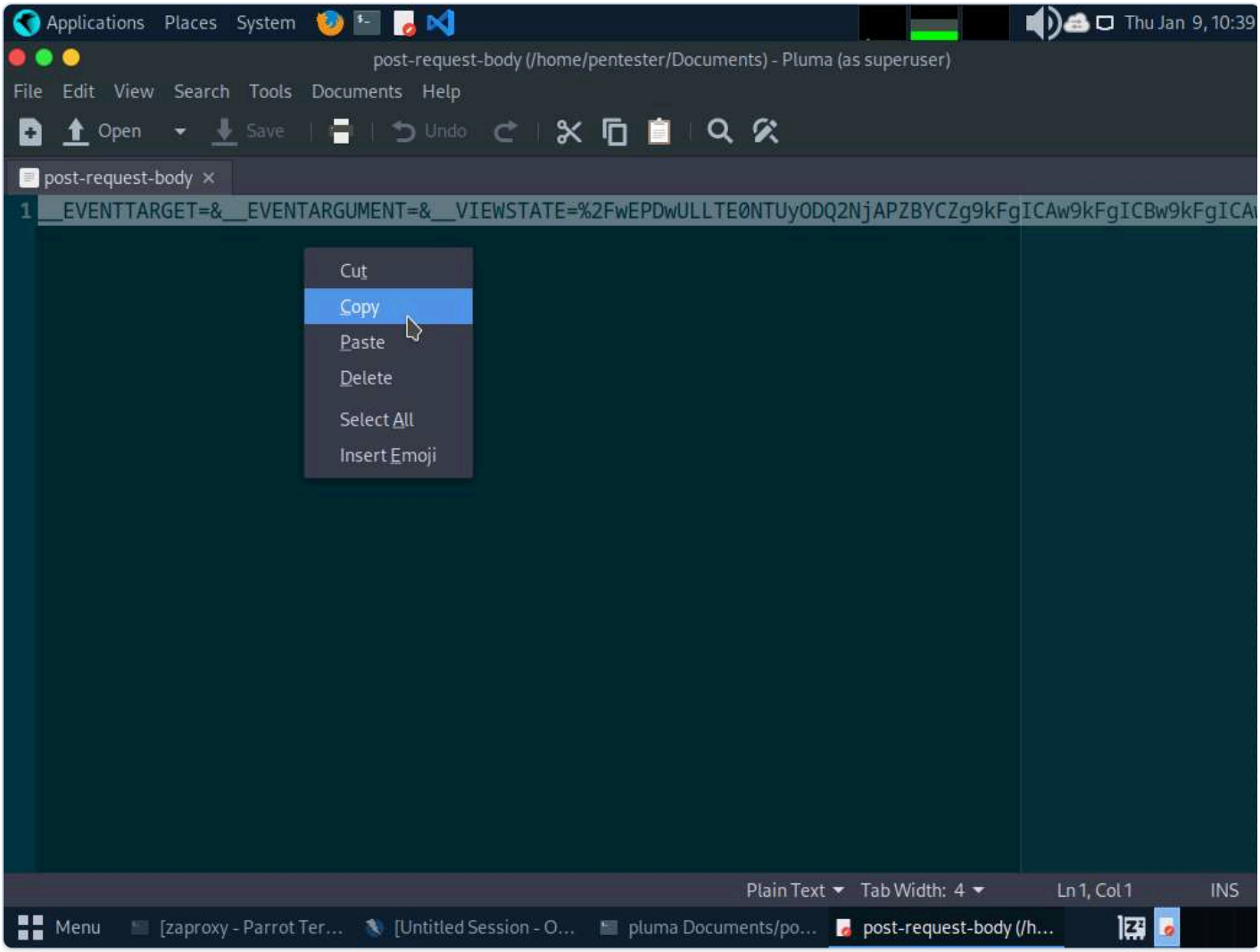
Task 1: Pentesting SQL Injection Vulnerability to Extract Database Information

In this task, we feed the saved POST request data into `sqlmap` to target the vulnerable `ctl00$BodyContentPlaceholder$txt_email` parameter, extract database structures, dump table contents from the `hotel` database, crack user hashes, and log into `www.luxurytreats.com`.

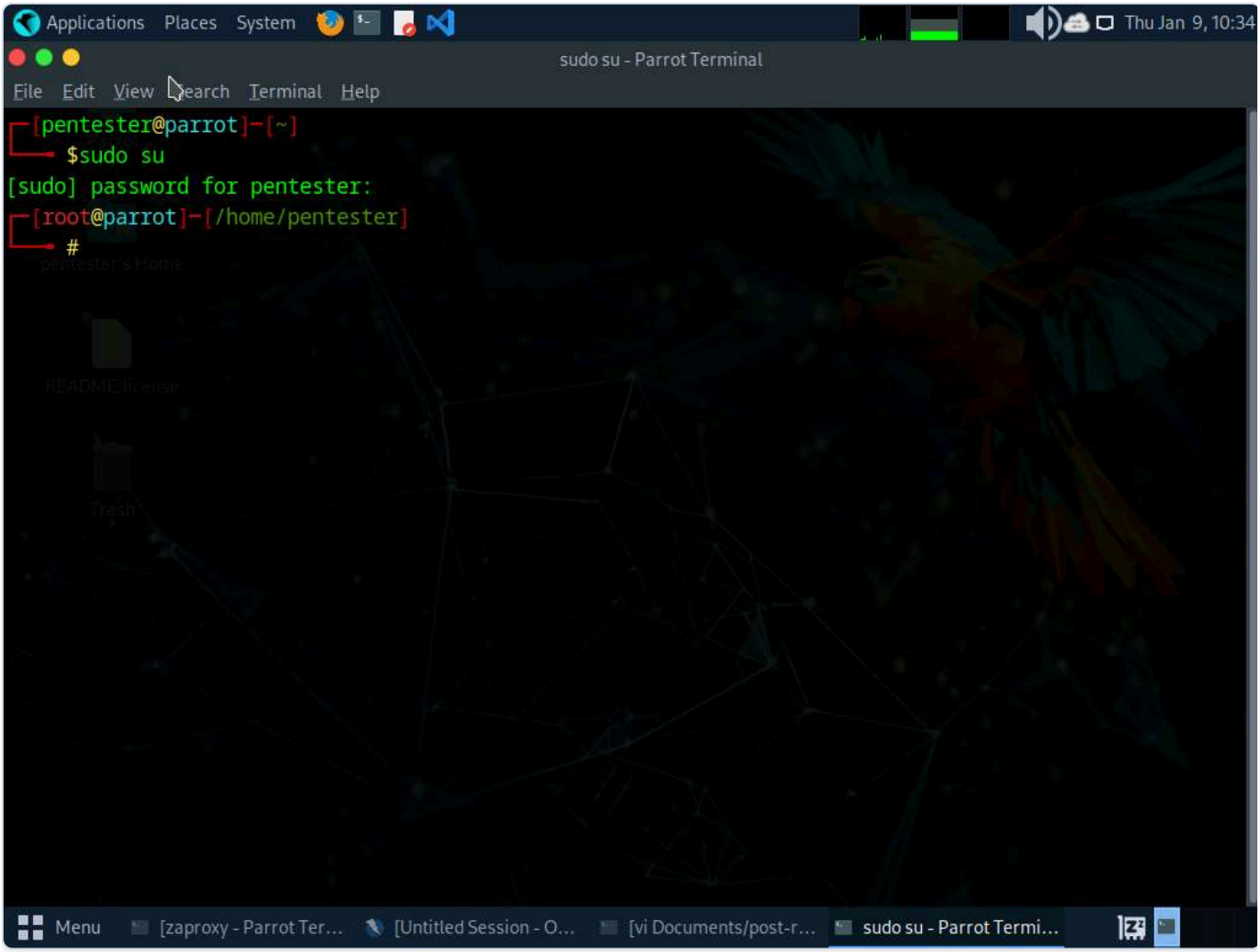
Lab Tasks

1. Click **Parrot Security**. Type **toor** in the Password field and press Enter.
2. Before starting, ensure you have the POST request body saved in the **post-request-body** file from Exercise 05.9.

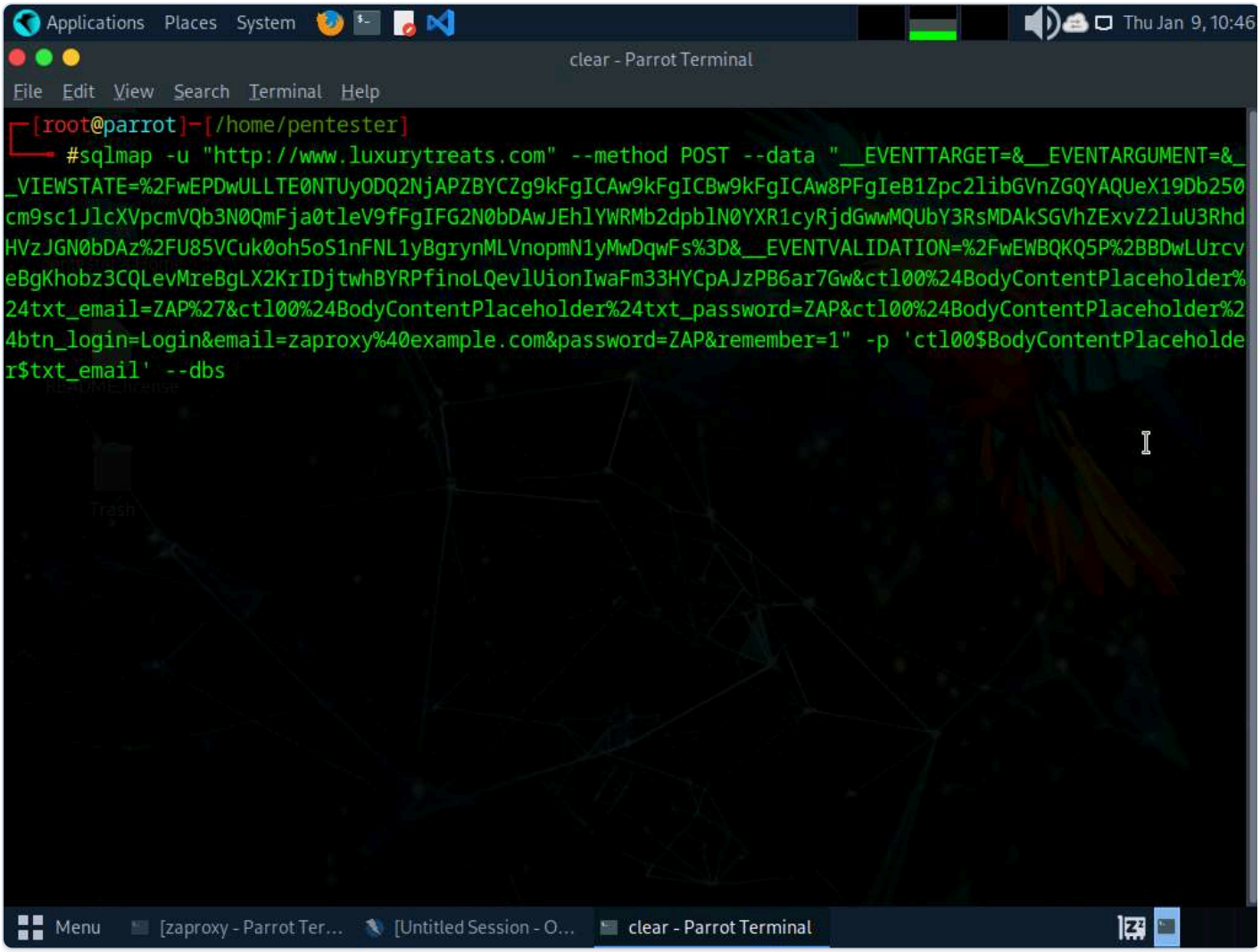
> **TIP:** 📁 **Local Repository Asset**: In your local environment, this extracted POST request body file is located at: `CPENTv2 Pen Testing Resources/CPENTv2 Scripts/Module 05 Web Application Penetration Testing/post-request-body`



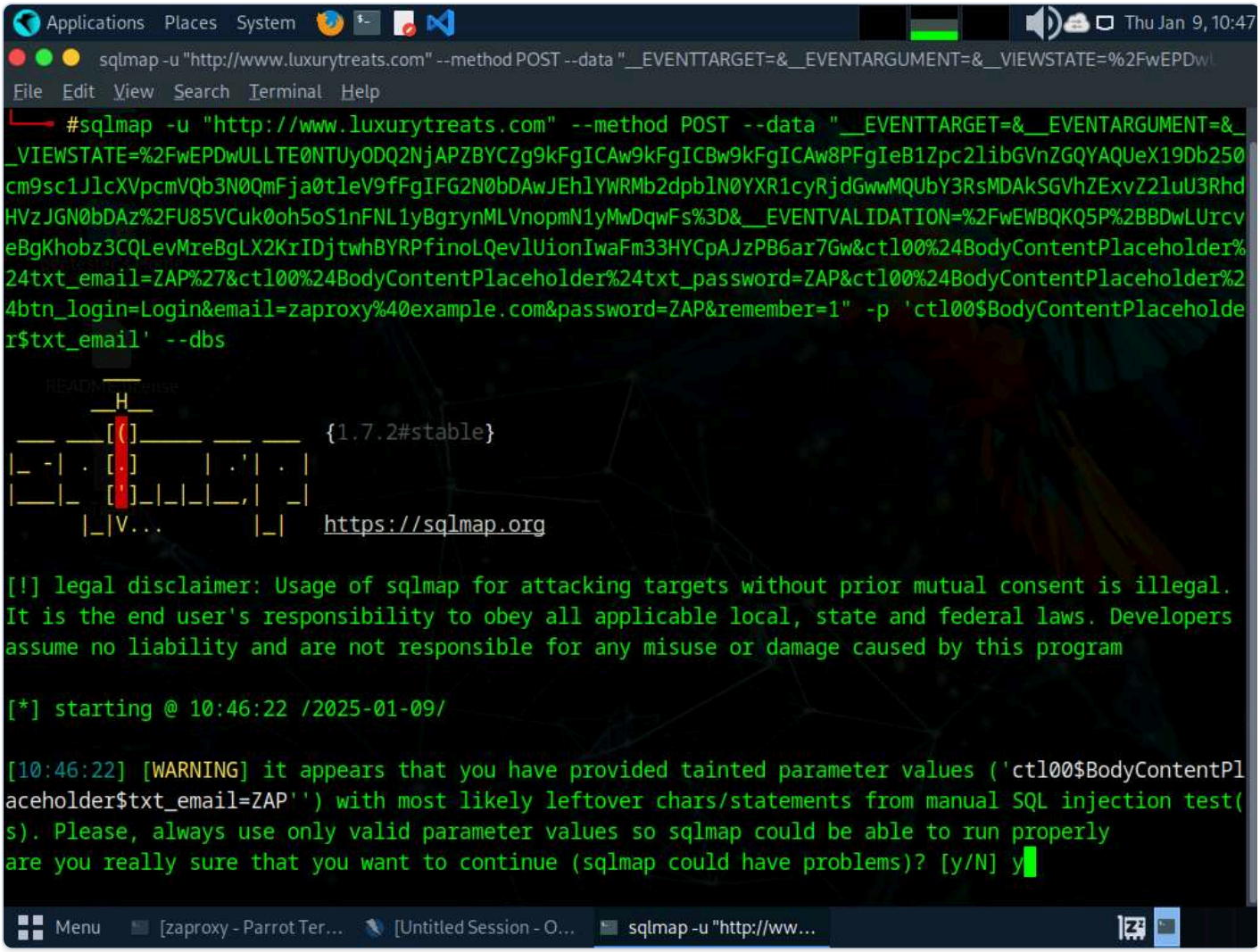
1. Launch a terminal, type **sudo su**, and press Enter. Enter password **toor**.



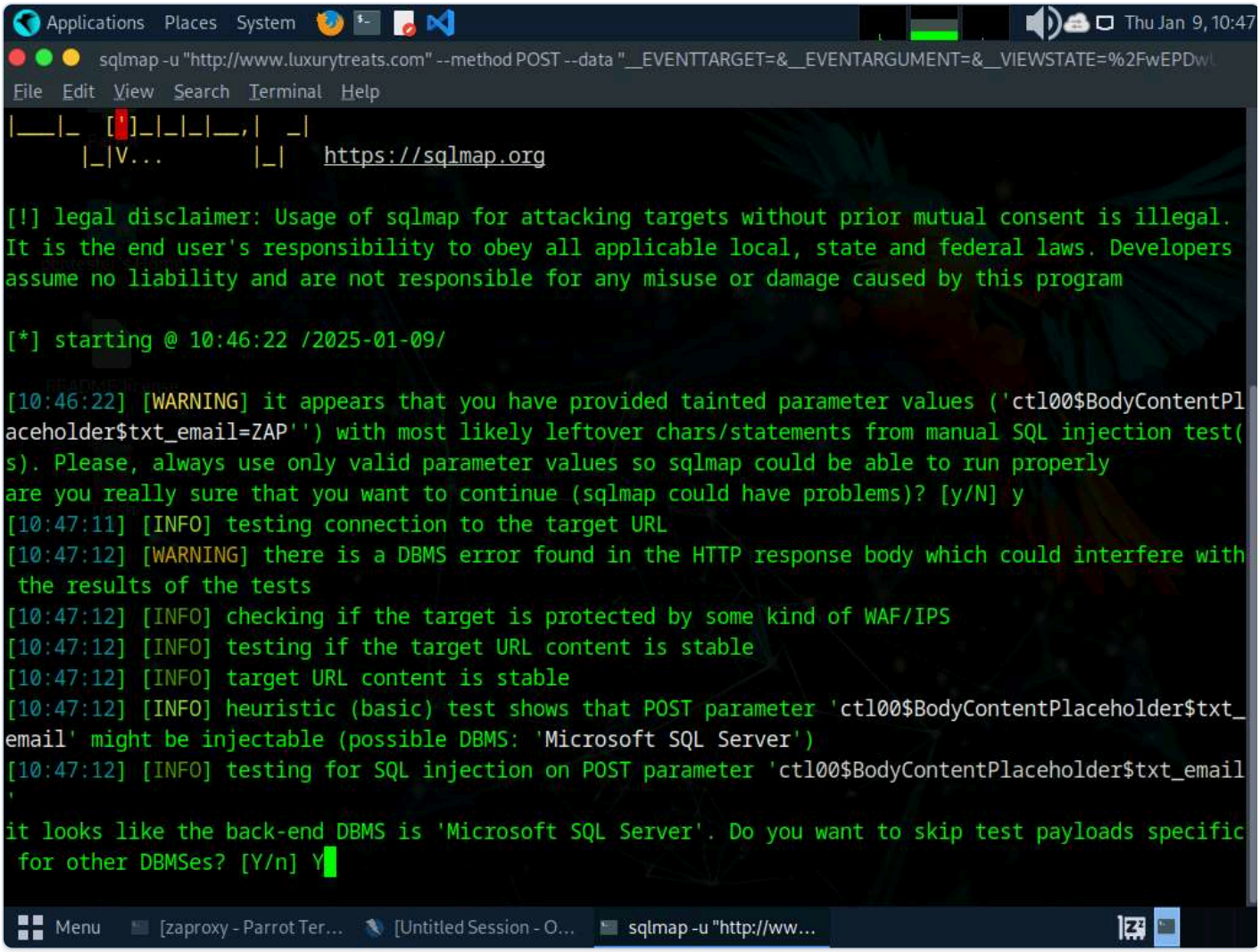
1. Use `sqlmap` to enumerate databases by supplying the target URL, POST data, and vulnerable parameter:
   sqlmap -u "www.luxurytreats.com" --method POST --data "[Copied POST Request]" -p 'ctl00$BodyContentPlaceholder$txt_email' --dbs



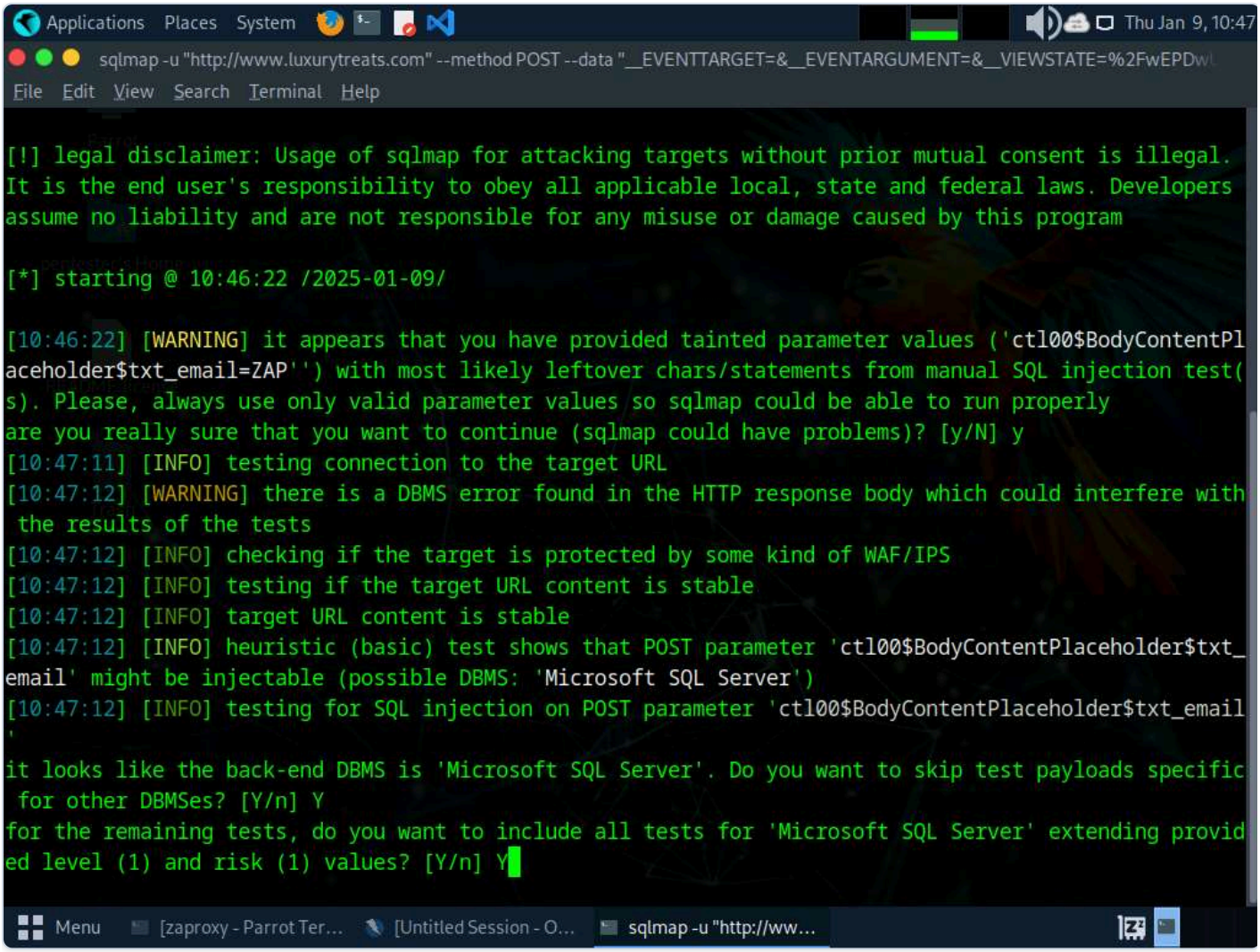
1. If `sqlmap` displays a warning stating that provided parameters are tainted, type **y** and press Enter.



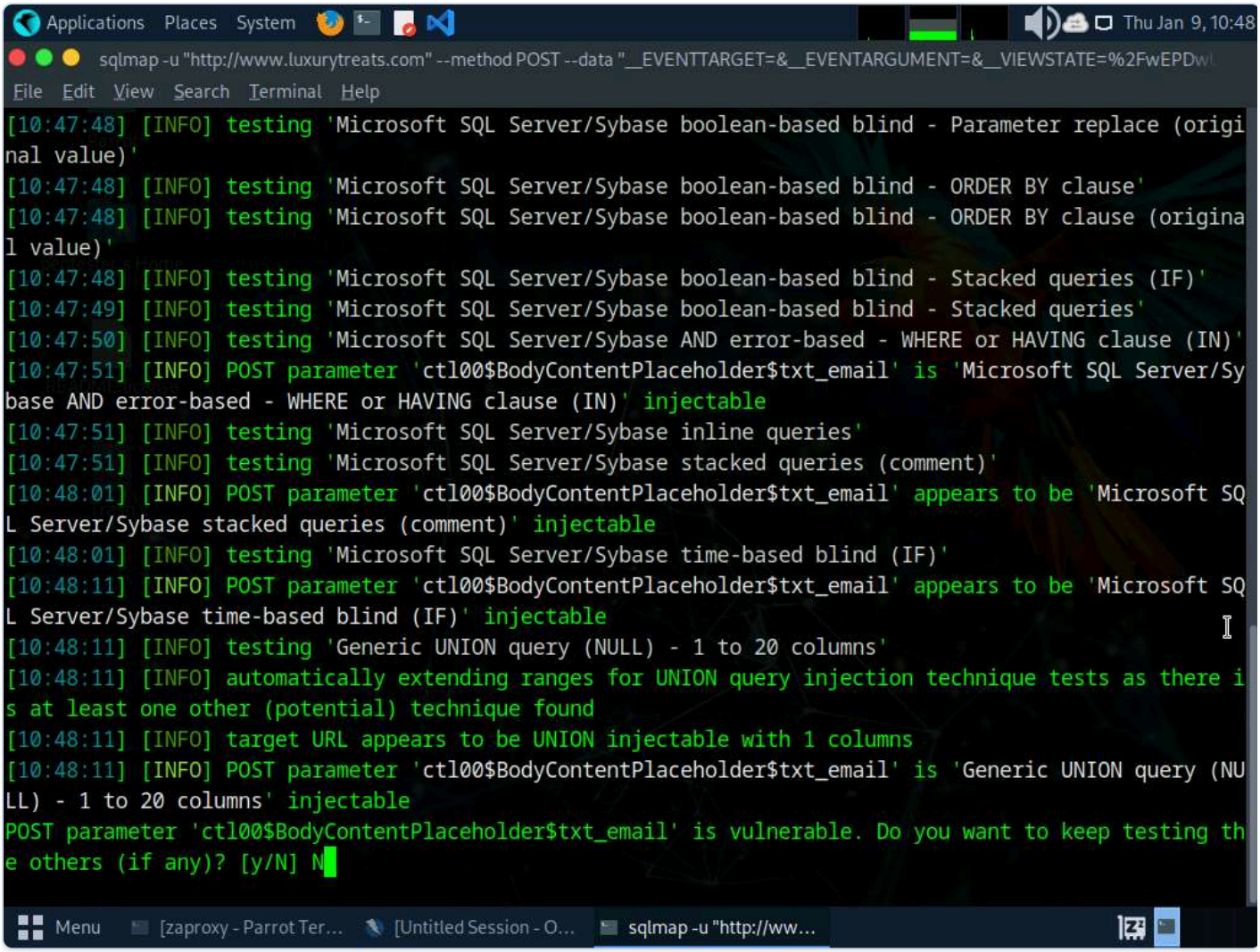
1. When `sqlmap` detects Microsoft SQL Server as the backend DBMS, type **Y** to skip tests for other DBMSes.



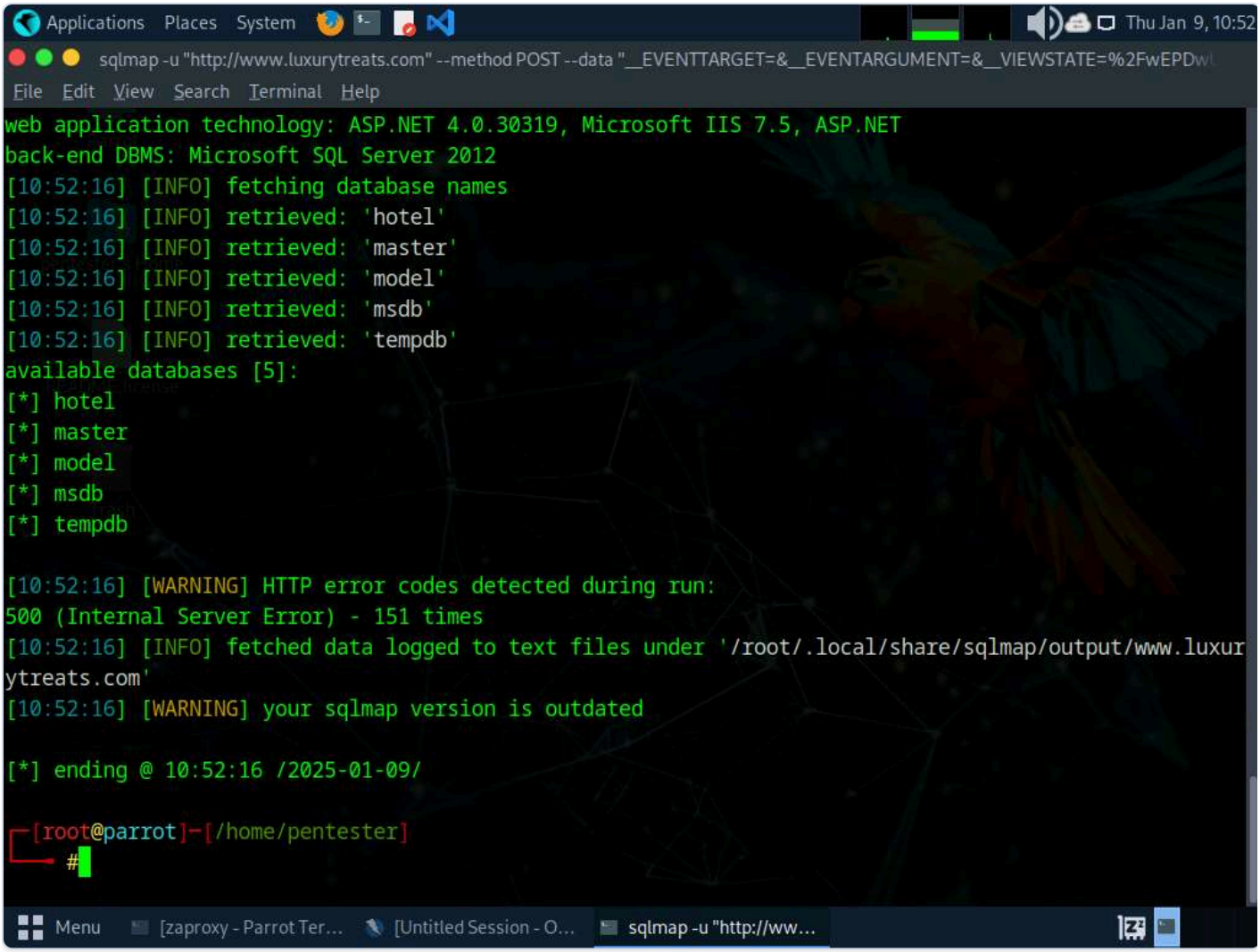
1. Type **Y** to include extended level 1/risk 1 tests for MS SQL Server.



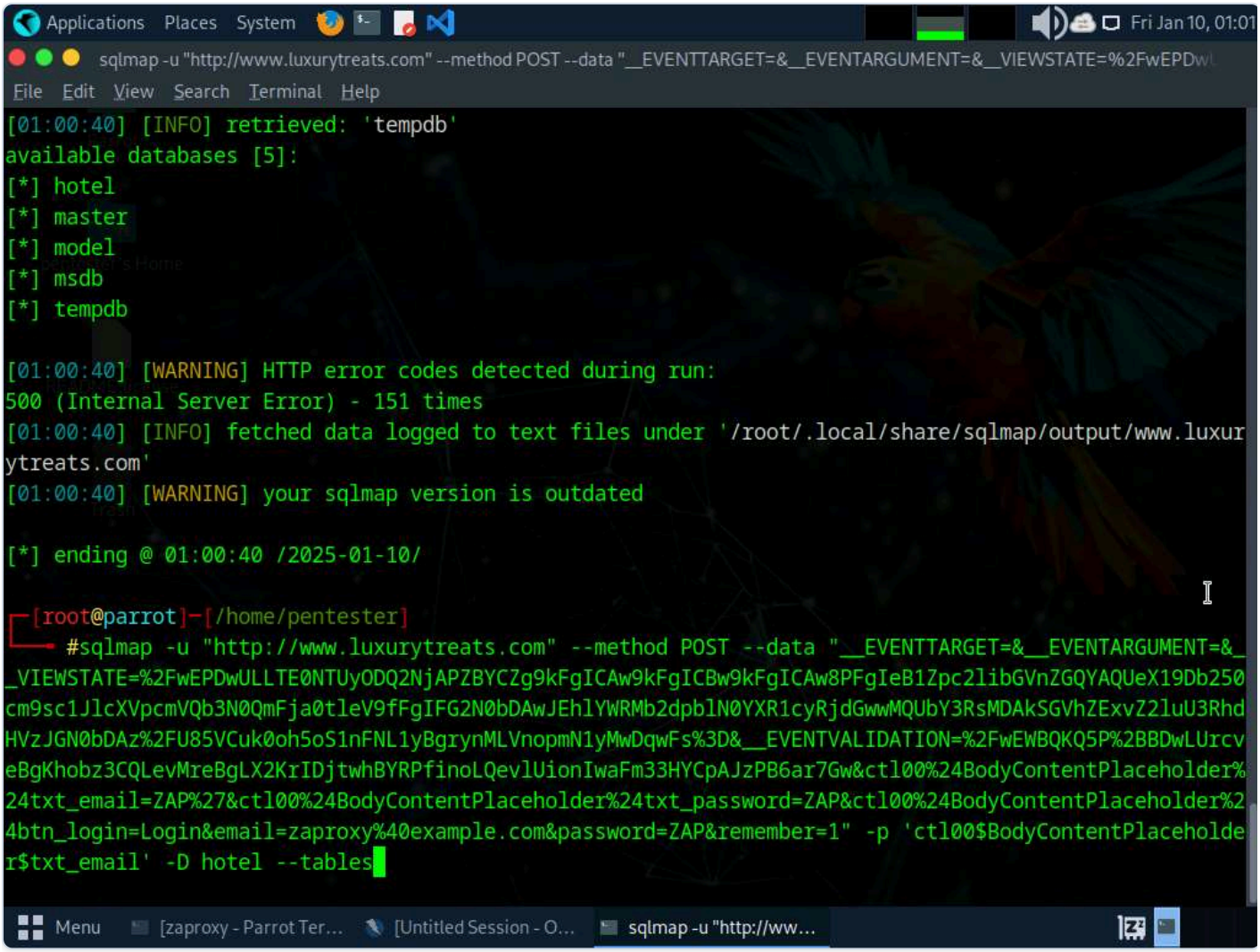
1. Once vulnerability is confirmed, type **N** to skip testing other parameters.



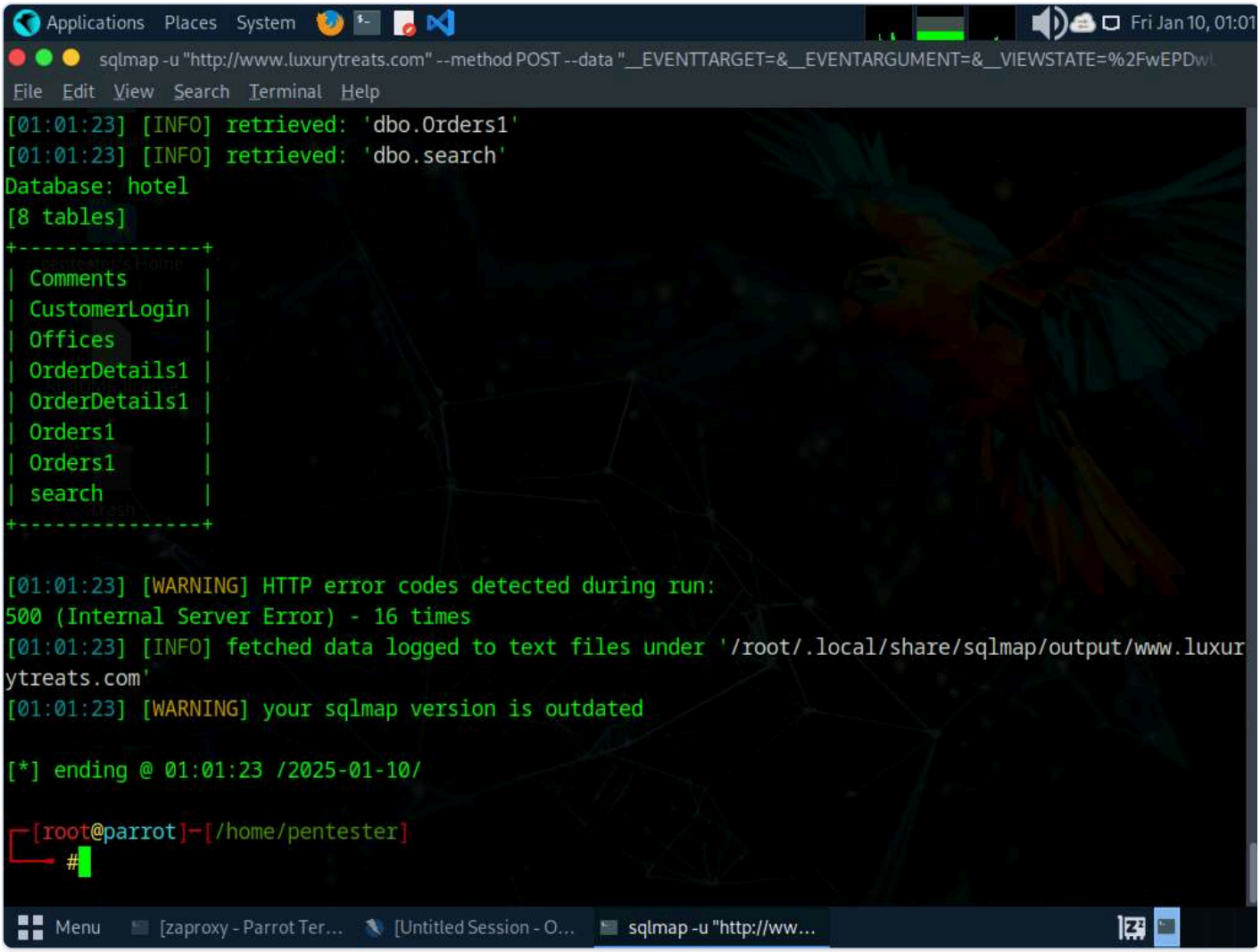
1. `sqlmap` extracts all available databases (including `hotel`).



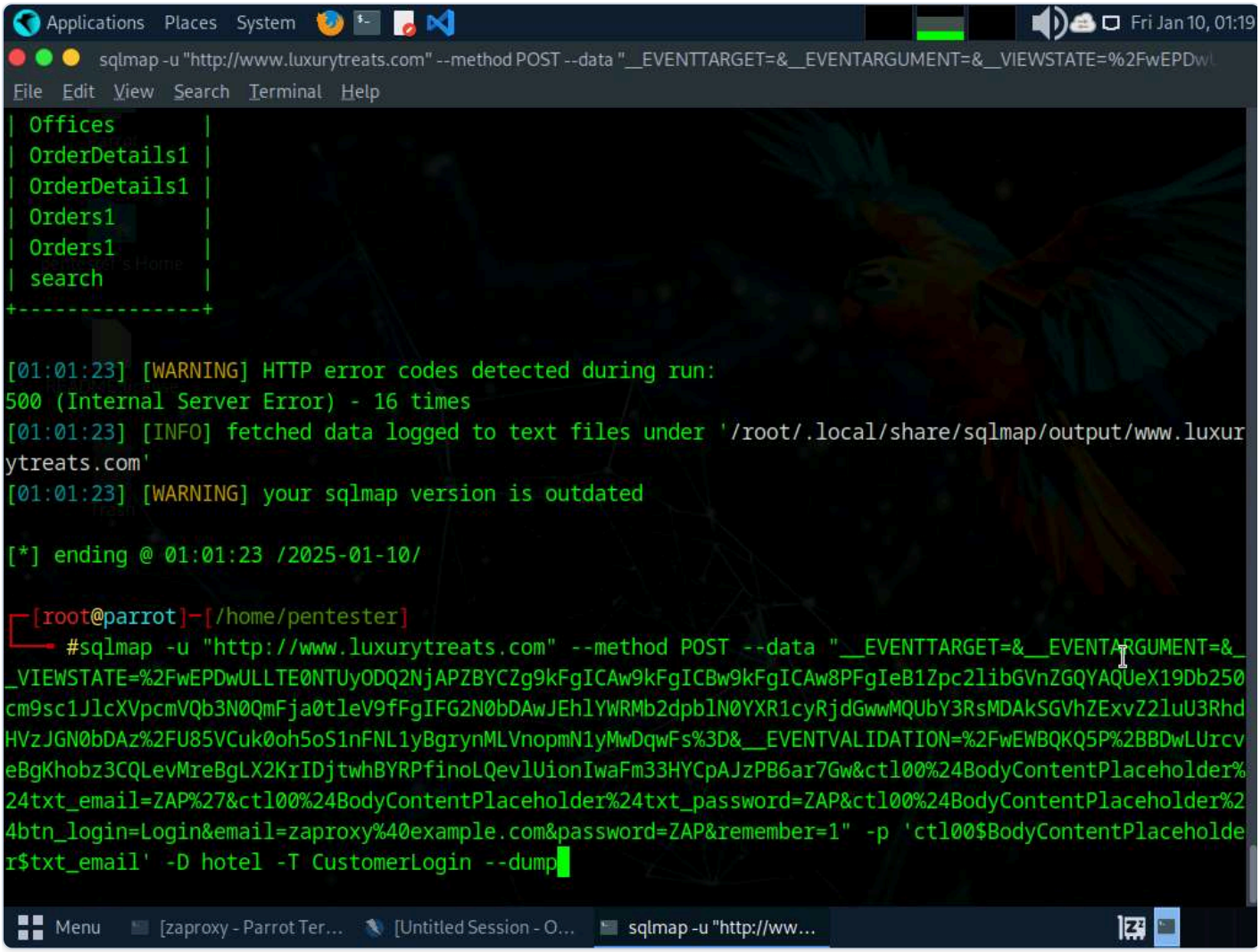
1. Enumerate tables inside the `hotel` database:
   sqlmap -u "www.luxurytreats.com" --method POST --data "[Copied POST request]" -p 'ctl00$BodyContentPlaceholder$txt_email' -D hotel --tables



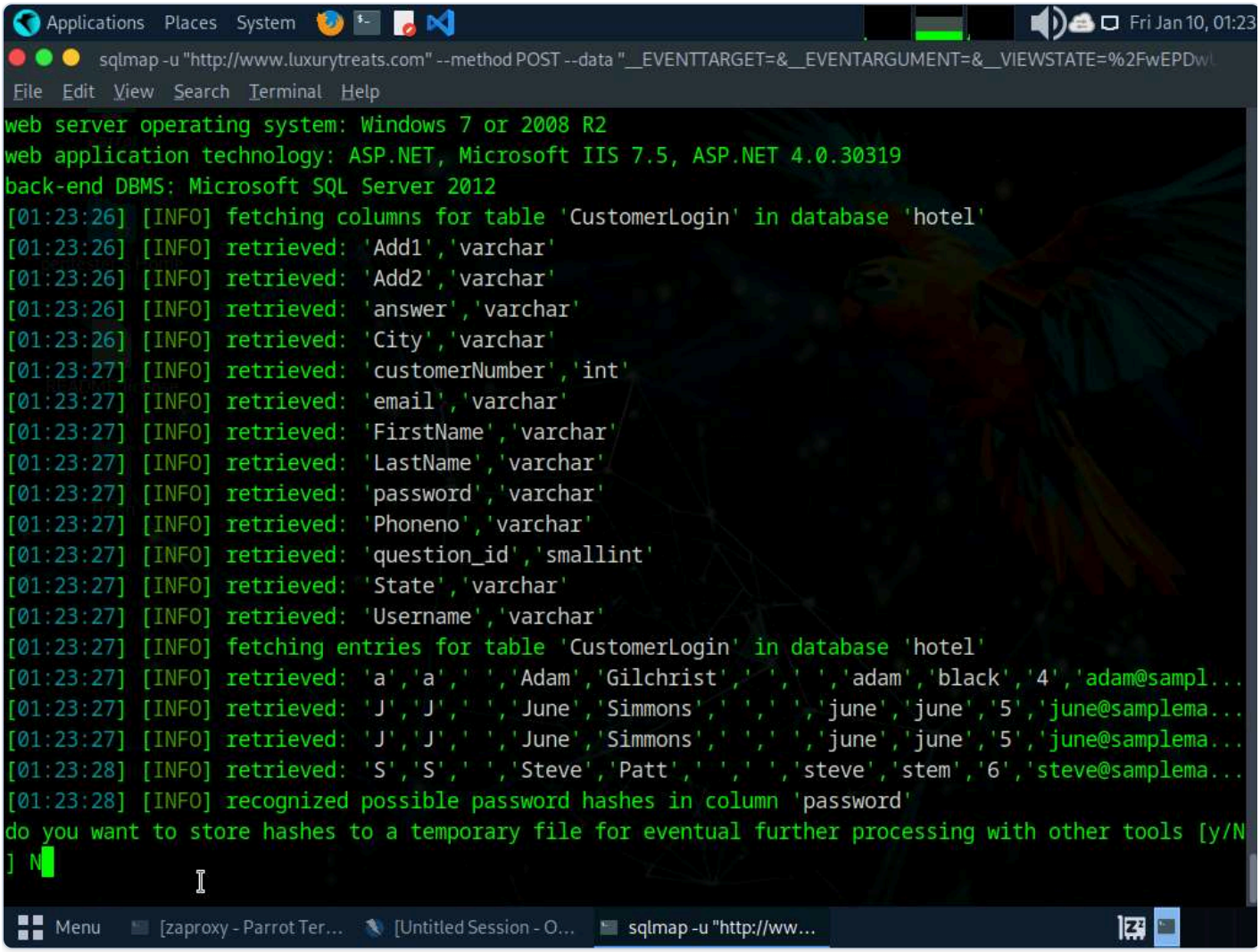
1. If prompted about tainted parameters, type **y** and press Enter.



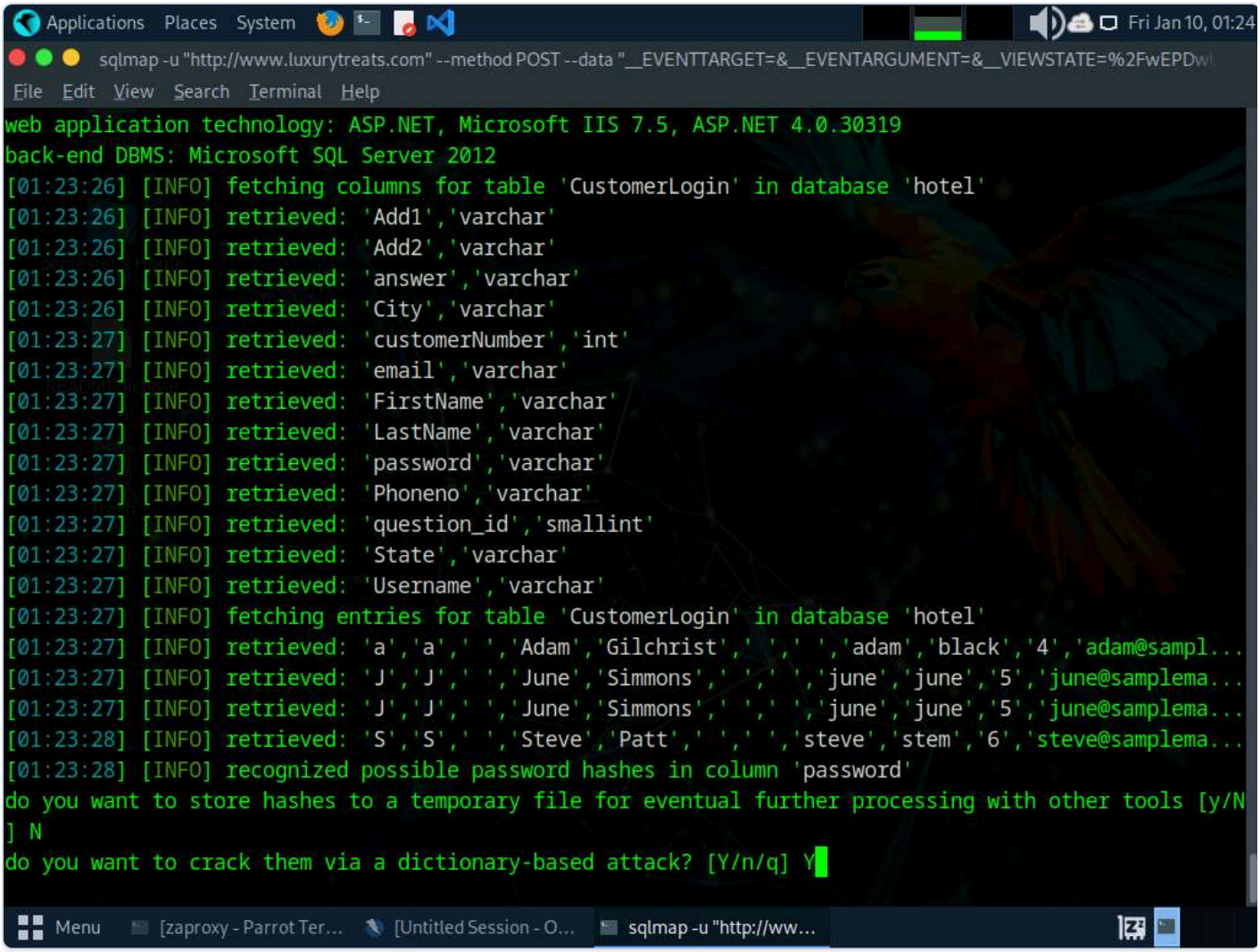
1. Dump all records from the **CustomerLogin** table:
   sqlmap -u "www.luxurytreats.com" --method POST --data "[Copied POST request]" -p 'ctl00$BodyContentPlaceholder$txt_email' -D hotel -T CustomerLogin --dump



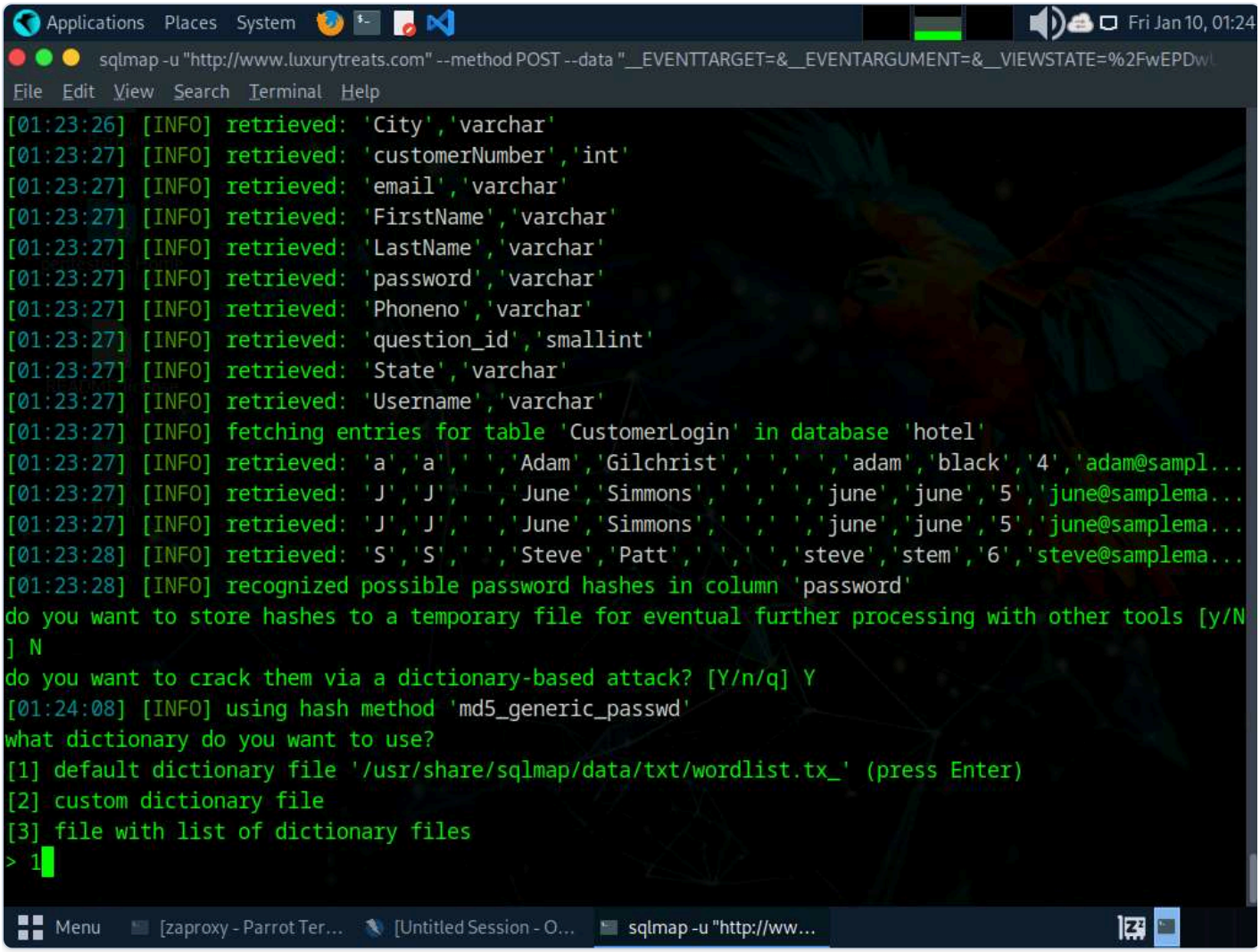
1. If prompted about tainted parameters, type **y**.
2. When asked whether to store hashes to a temporary file, type **N**.



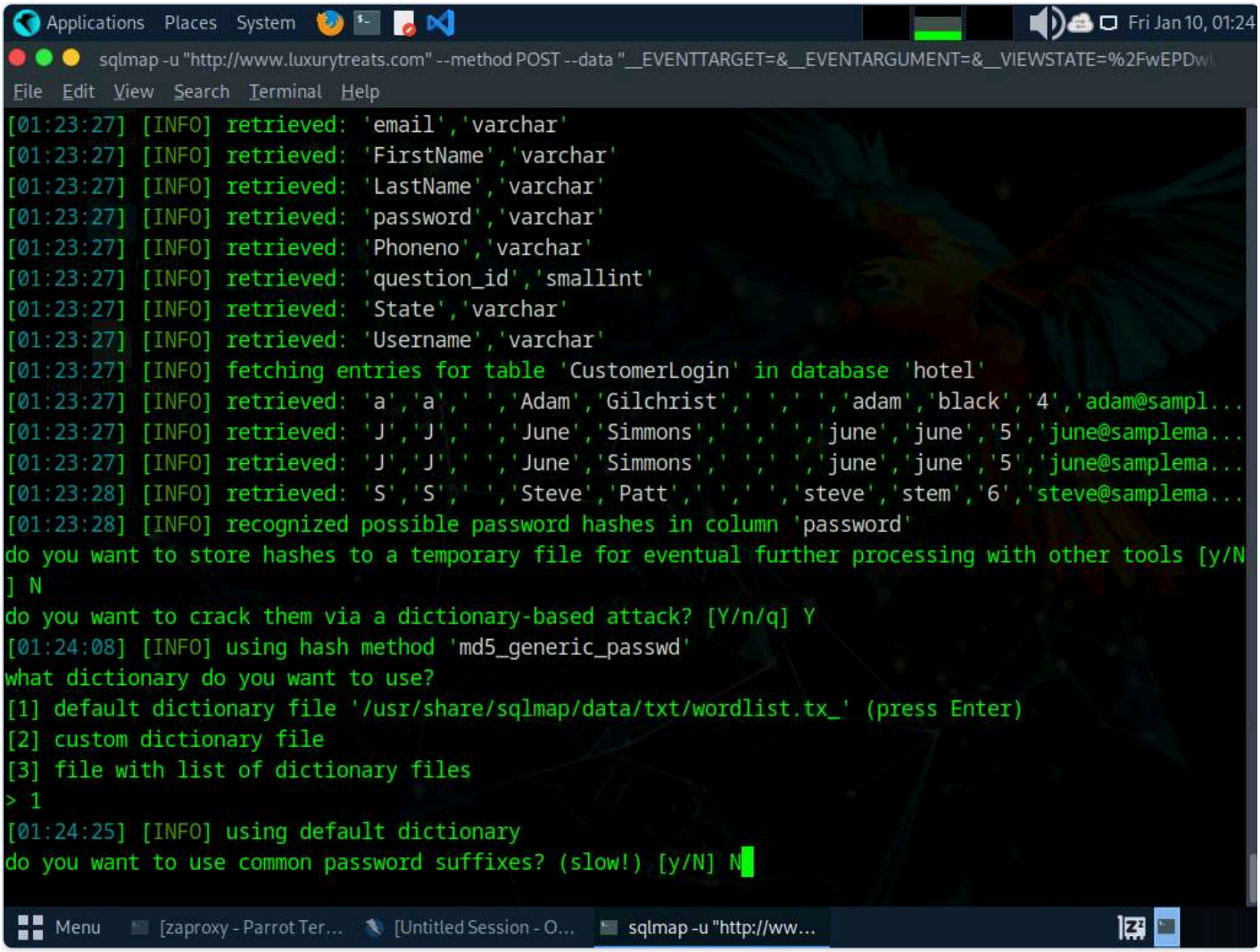
1. Type **Y** to attempt cracking password hashes using a dictionary attack.



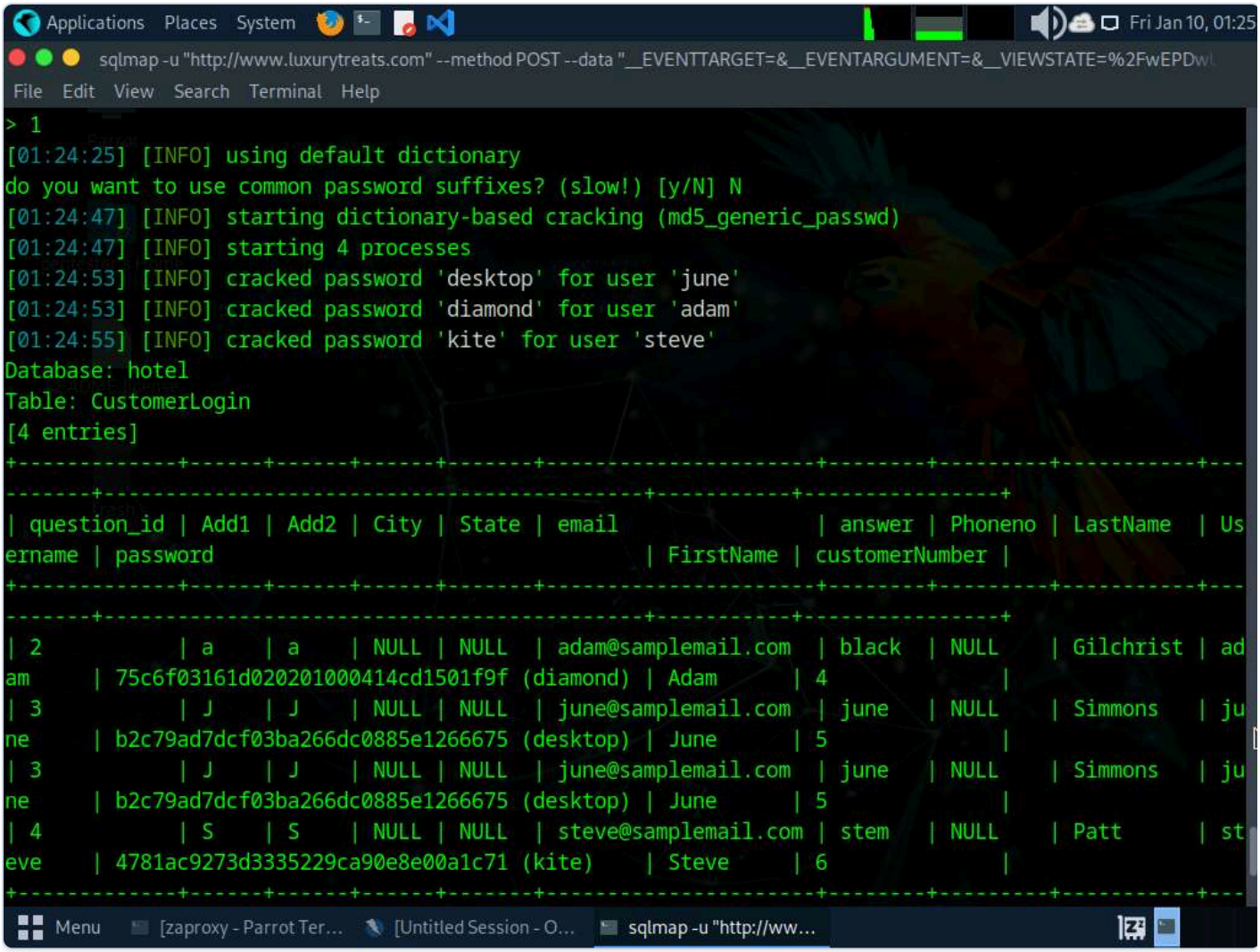
1. Type **1** to select the `sqlmap` default wordlist.



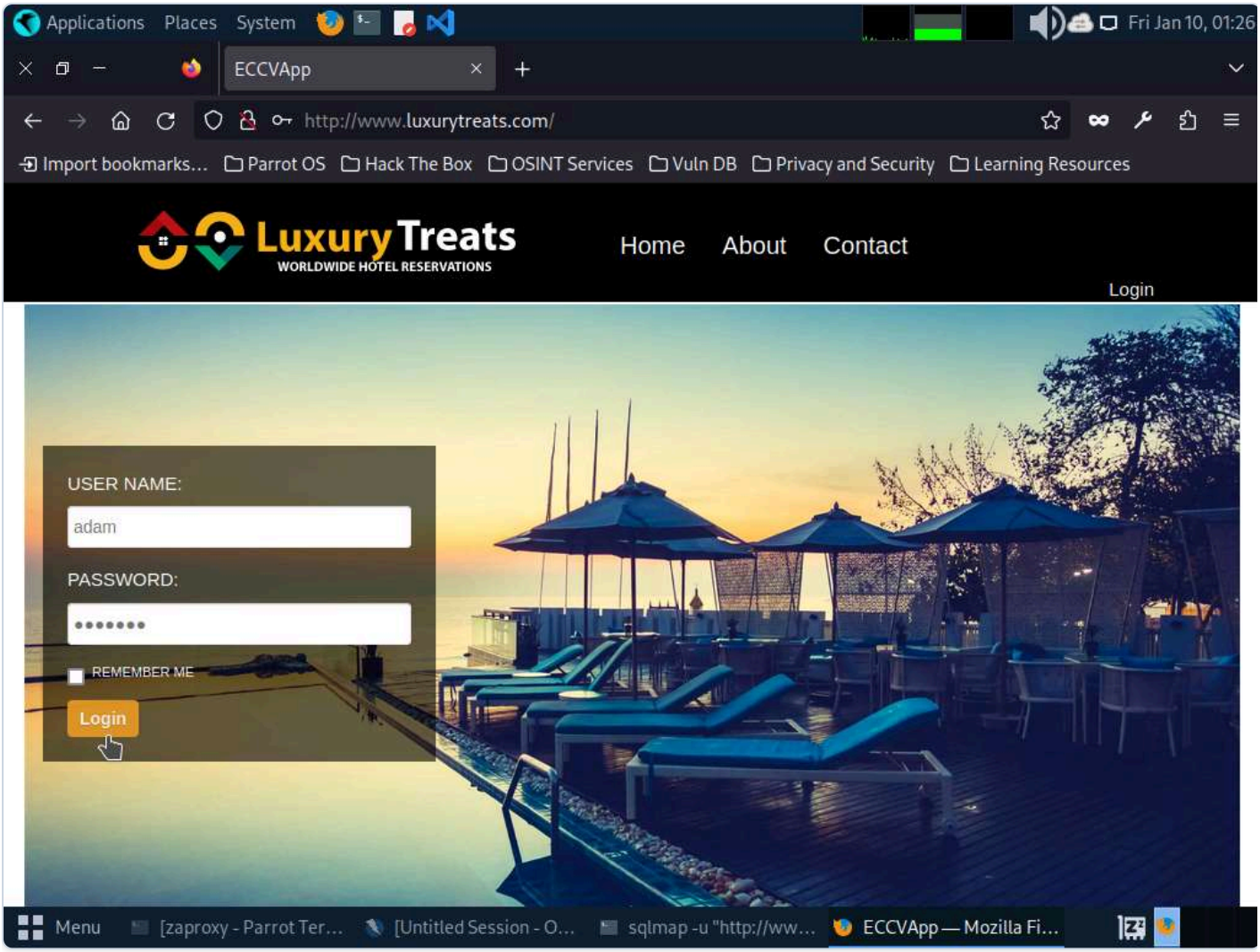
1. Type **N** when prompted regarding common password suffixes.



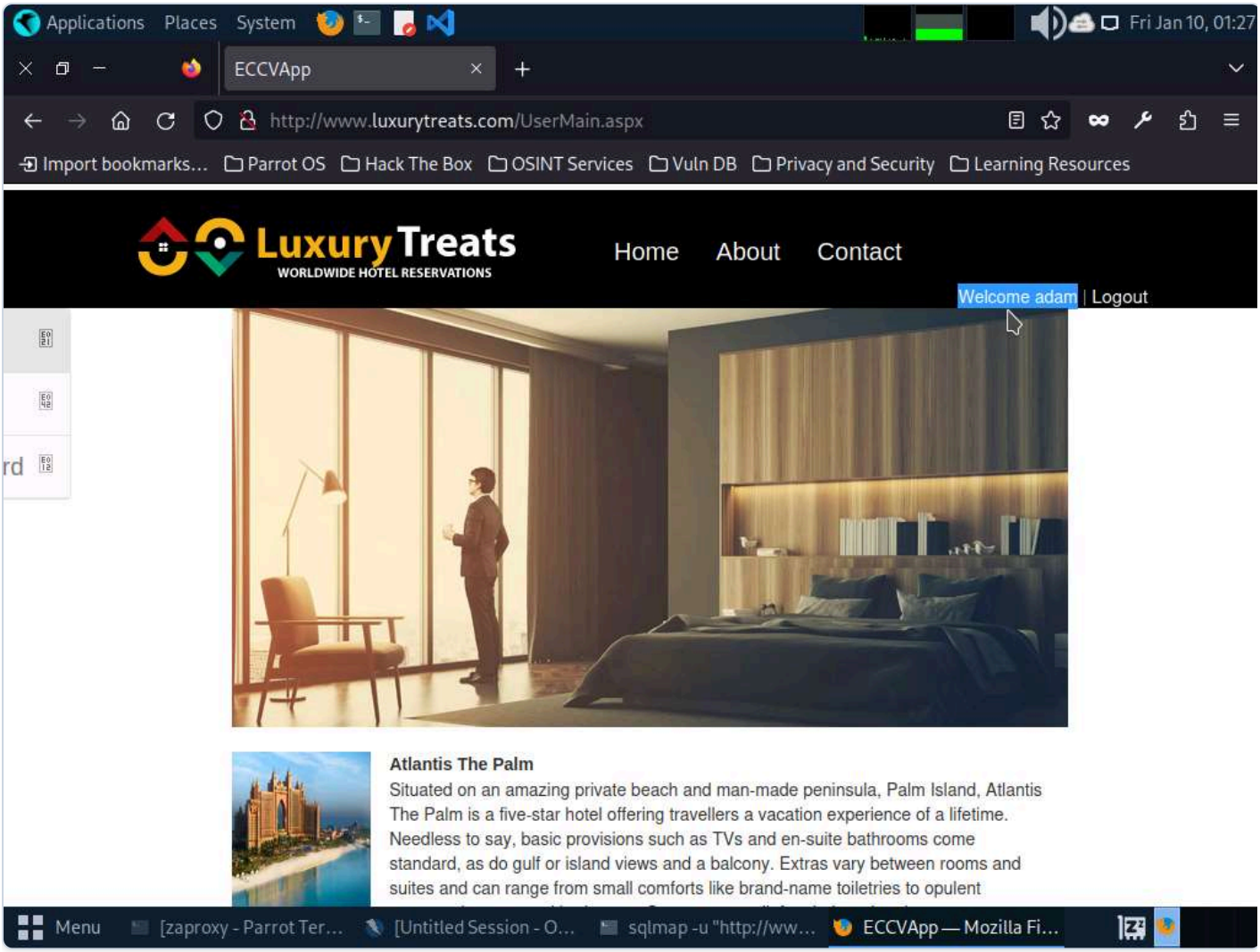
1. `sqlmap` successfully cracks the password hashes (e.g., username **adam** with password **diamond**).



1. Open Firefox, navigate to **http://www.luxurytreats.com**, and log in with username **adam** and password **diamond**.



1. Confirm successful authentication to the web application.



1. Close all open windows.

This concludes the lab exercise.

IMPORTANT: ❓ **Question 5.10.1.1** On the Parrot Security machine, utilize SQLMap in conjunction with the previously copied POST request from the `www.luxurytreats.com` website to retrieve database information from the target web application. Specifically, extract the "Offices" table from the database and determine the number of offices located in the United States. Document the findings accordingly. [!SUCCESS] ✅ **Answer:** **3**